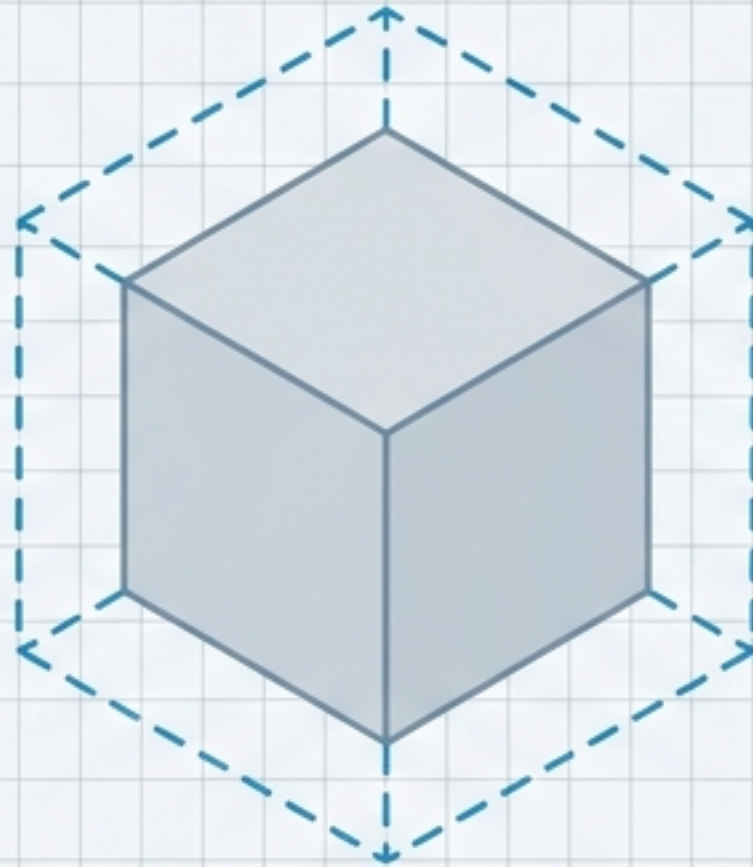


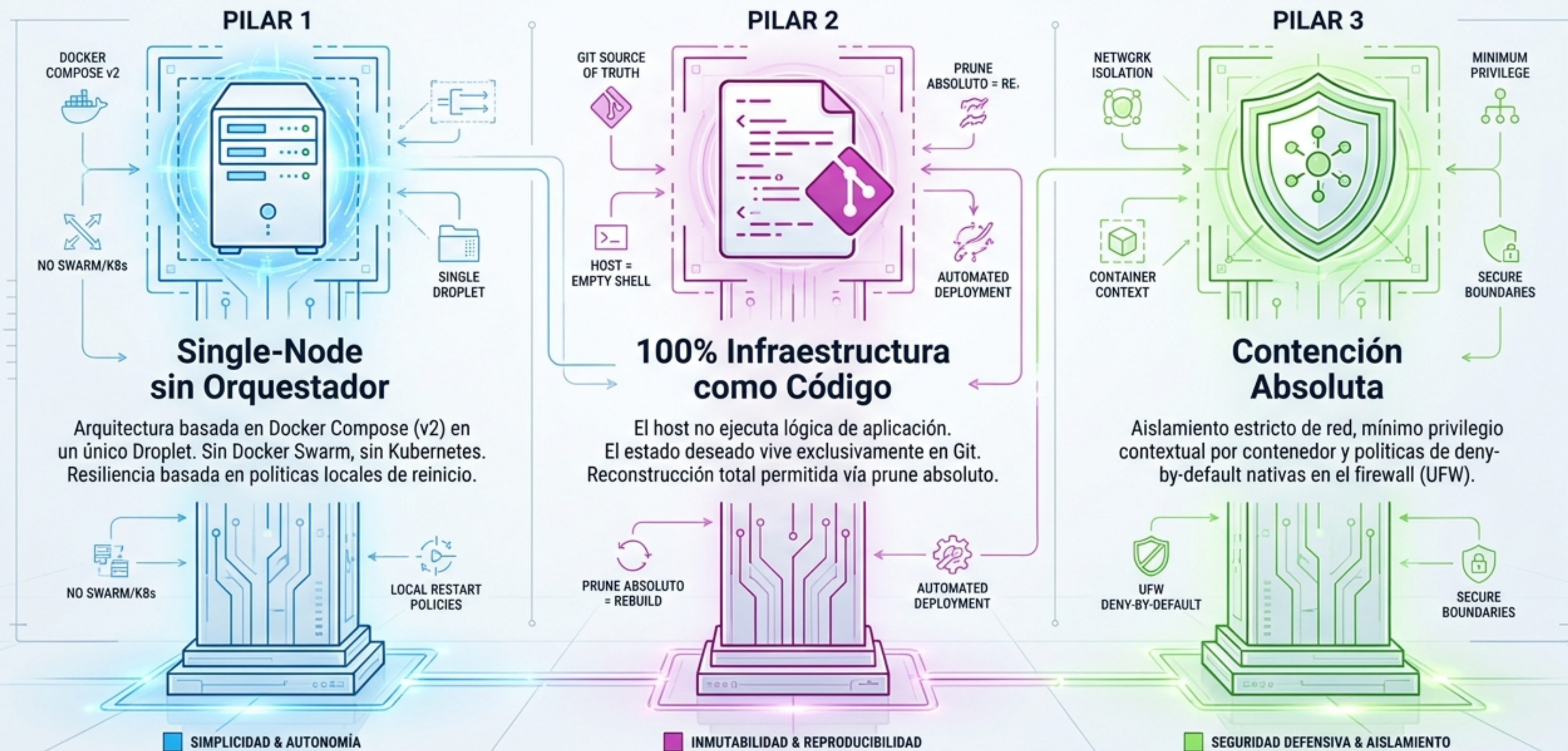


Blueprint Arquitectónico: server_monitoring_2602

Modelo Normativo IaC, Seguridad Runtime y Resiliencia Single-Node

THE LIGHT CLEAN ROOM TECHNICAL BLUEPRINT: INFRAESTRUCTURA DE ALTA RESILIENCIA Y SEGURIDAD

Principios Fundamentales - Diseño de Tres Pilares para Arquitectura Aislada

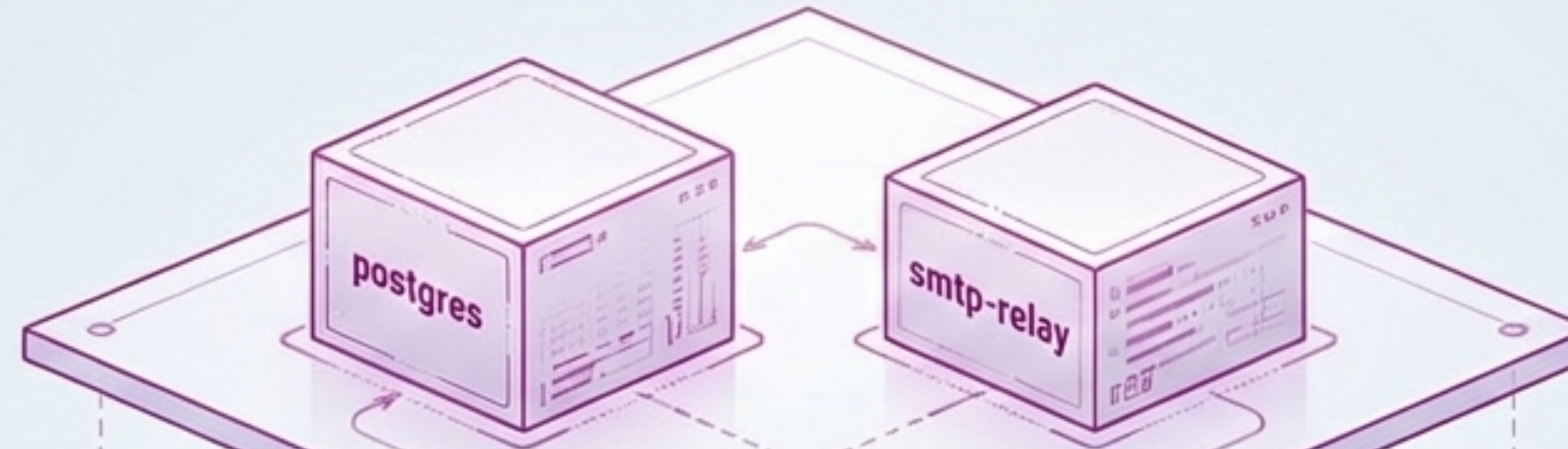


THE LIGHT CLEAN ROOM TECHNICAL BLUEPRINT: INFRAESTRUCTURA DE ALTA RESILIENCIA Y SEGURIDAD

Principios Fundamentales - Diseño de Tres Planos para Arquitectura Aislada

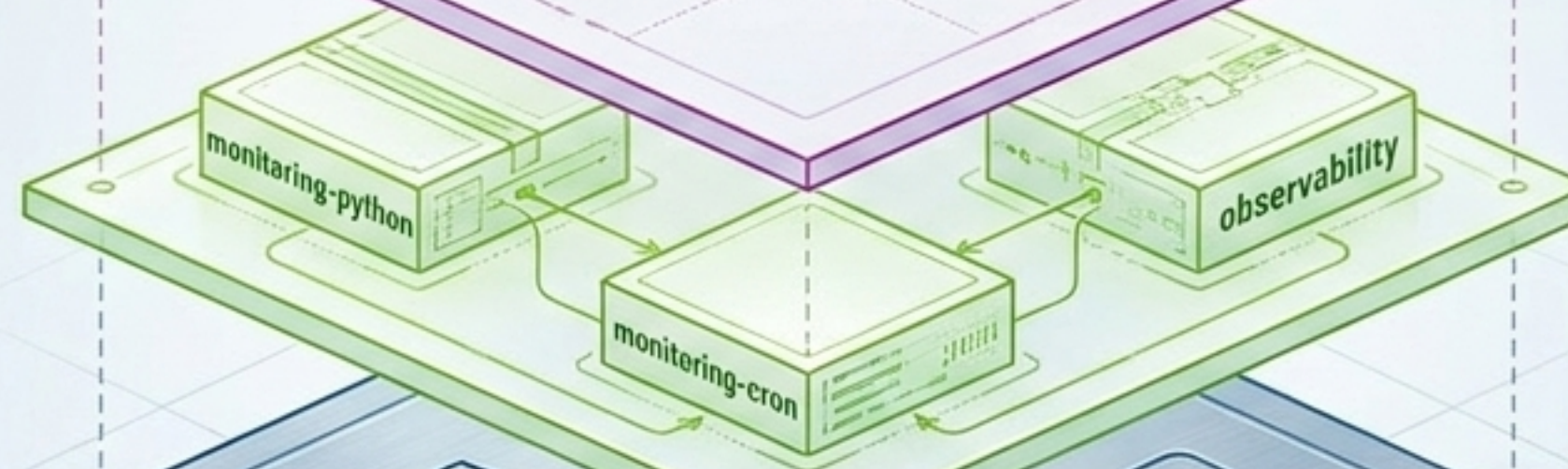
PLANO 1: Micro-Stacks

Autonomía exportable.
Bases de datos y servicios de negocio independientes.



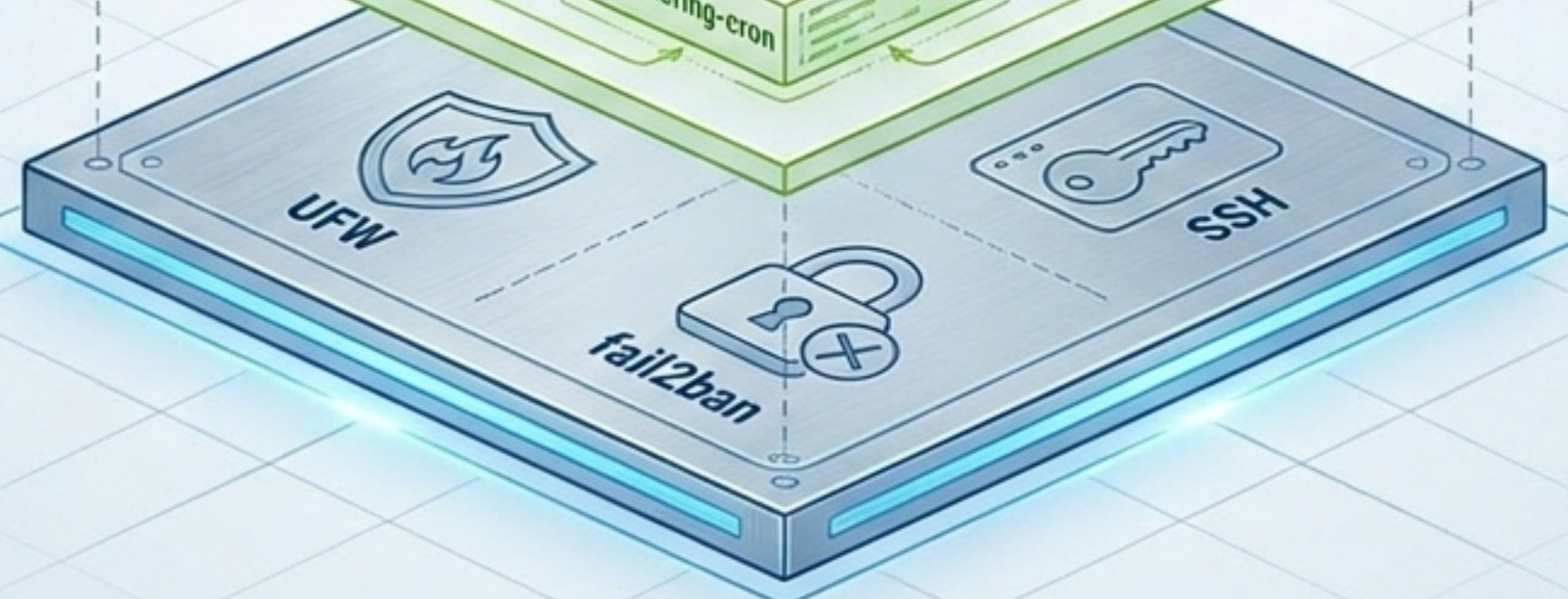
PLANO 2: Infra-Stacks

Runtime interno.
Tareas programadas, ingestión de logs y alertas.



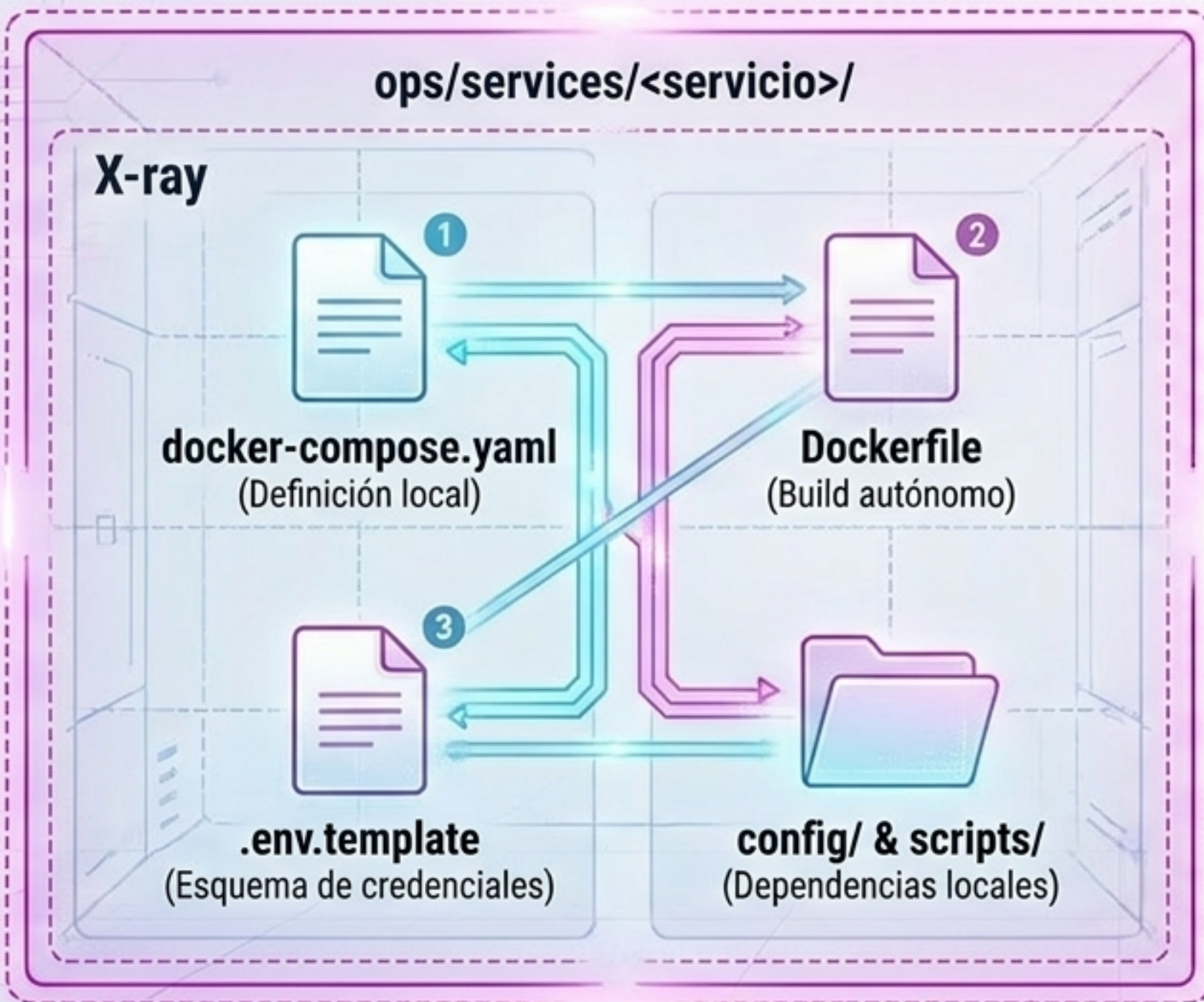
PLANO 3: Seguridad del Host

OS nativo Ubuntu 24.04.
Orquestación y defensa perimetral estricta.



REGLA: Ningún plano introduce dependencias cruzadas estructurales en los demás.

El Modelo de Micro-Stacks Autónomos

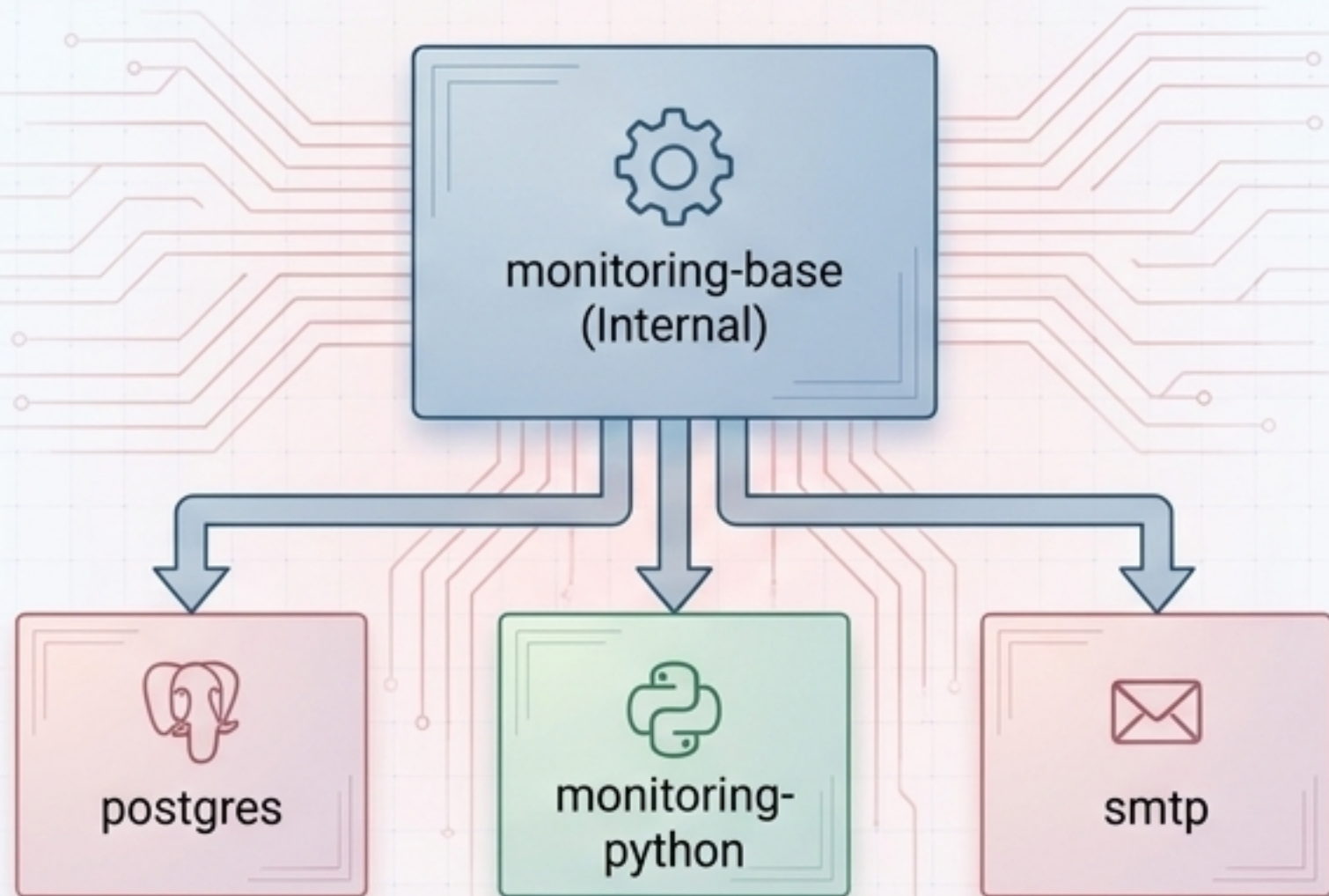


Anatomía de Autonomía (ADR-0008)

- Un Micro-Stack debe poder copiarse a otro repositorio sin perder funcionalidad.
- Prohibido depender de bind mounts absolutos del host.
- Prohibido depender de archivos fuera de su propio directorio.
- Garantiza portabilidad y evita acoplamiento estructural.

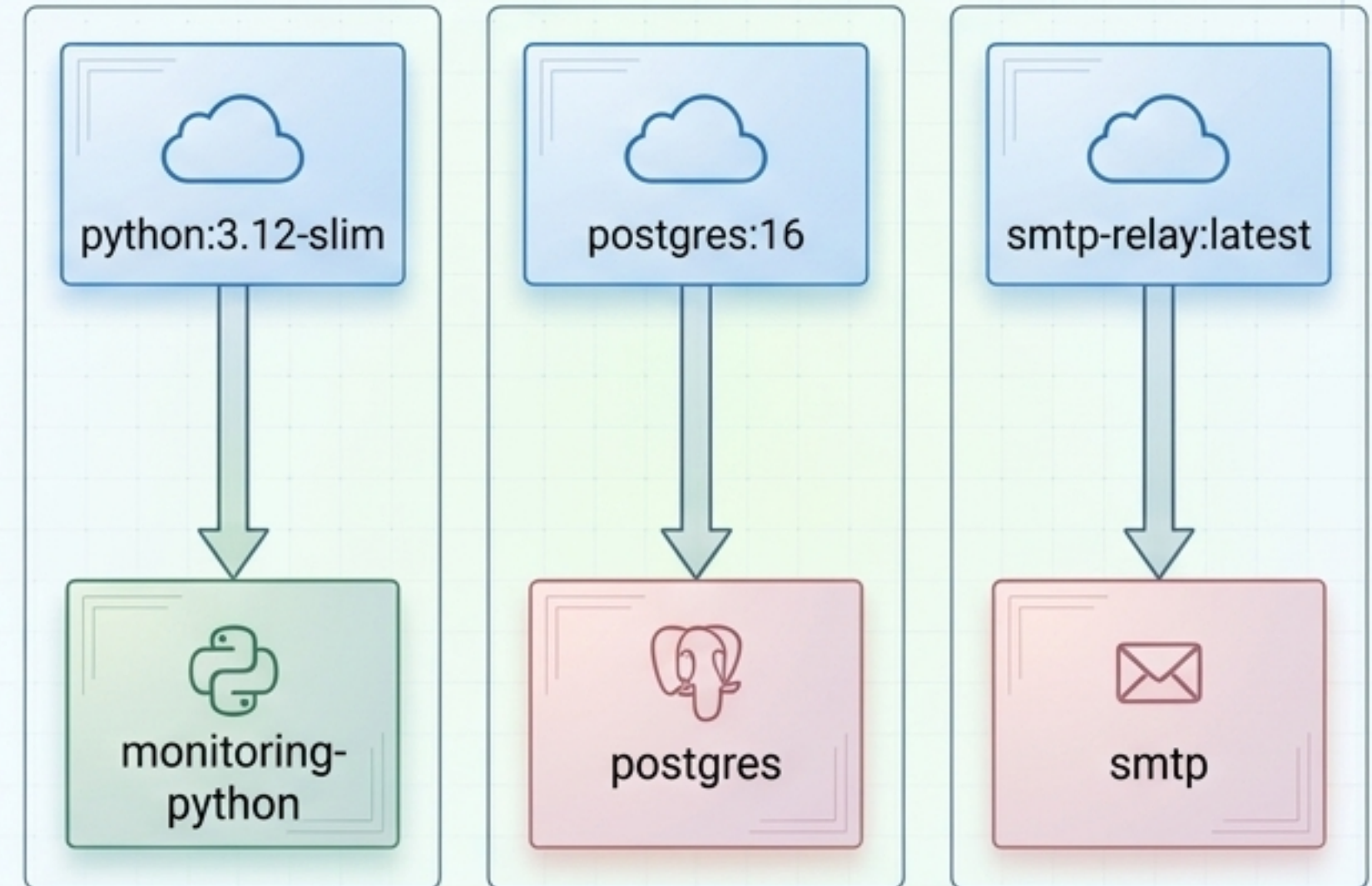
Gobernanza de Imágenes Base

✗ Antipatrón: Acoplamiento Centralizado



Atar todos los servicios a un monorepo interno crea **dependencias frágiles** y rompe la portabilidad del micro-stack.

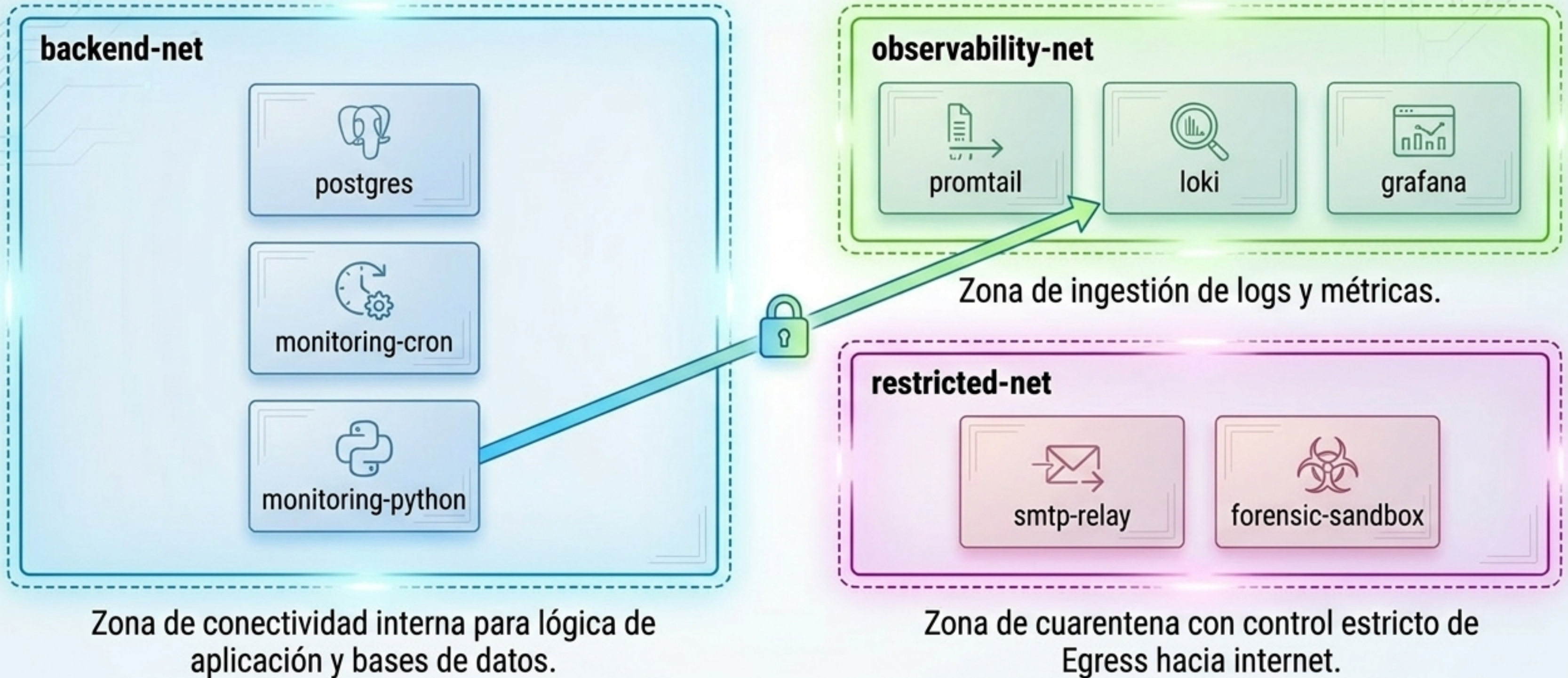
✓ Normativa: Autonomía de Extensión



Cada servicio extiende una imagen externa oficial de forma independiente. La **autonomía prevalece sobre la reutilización**.

Aislamiento de Redes (Zero Global Network)

Eliminación de red global (ADR-0021). Segmentación estricta por dominios.



Política Estricta de Exposición de Puertos

Nivel 3: Exposición Global

```
ports: ['3001:3001']
```

Prohibido. Bypassa UFW y expone el servicio directamente a internet (0.0.0.0).



Nivel 2: Acceso Local (Loopback)

```
ports: ['127.0.0.1:3001:3001']
```

Permitido solo para interfaces de debug (ej. Grafana). Aislado del exterior.



Nivel 1: Resolución Interna

```
Sin directiva 'ports'
```

La norma. Comunicación exclusiva mediante el DNS interno de las redes Docker.

Container Network (Interna)



Ejecución de Código y Cronjobs



Host - Plano 3

Sistema Operativo Anfitrión

- Solo gestiona el runtime Docker y UFW.
- Permitido: logrotate, fail2ban, OS updates.
- El host NO ejecuta lógica de aplicación.
- Prohibido el uso de wrappers bash.

Contenedor - Plano 2

Contenedor "monitoring-cron"



- Actúa como el scheduler oficial del sistema.
- Ejecuta los módulos Python (python3 -m paquete).
- Invoca utilidades operativas y backups (pg_dump).
- Garantiza un entorno reproducible e independiente.

REGLA: Todo código de aplicación vive y se ejecuta dentro de los entornos reproducibles de los contenedores, jamás en el Host.

Gestión Minimalista de Credenciales

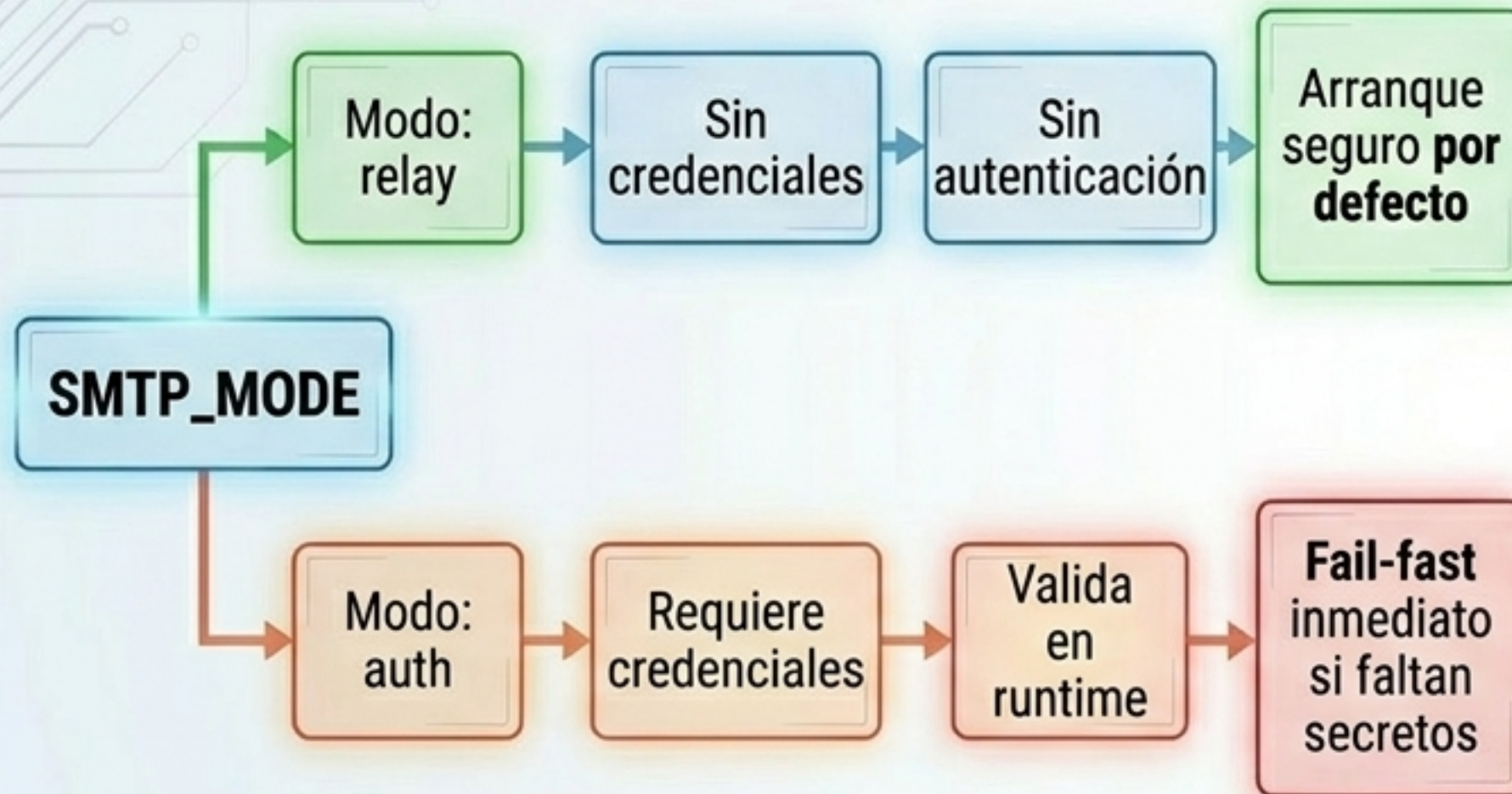
Modelo Single-Node sin orquestadores pesados (ADR-0013)



Se elimina el uso de carpetas `'secrets/'` locales y la dependencia de Docker Swarm, garantizando una arquitectura ligera y portátil.

Inicialización de Servicios y Fail-Fast

Desacoplamiento SMTP (ADR-0025, ADR-0005)



```
# PROHIBIDO: Carga de secretos al importar
import secrets
db = connect_database(secrets.get_pass())
```

```
# OBLIGATORIO: Inicialización explícita
if __name__ == '__main__':
    init_config()
    db = connect_database()
```

Prohibidos los side-effects en import-time. Todo entrypoint debe invocar **init_config()** explícitamente en tiempo de ejecución.

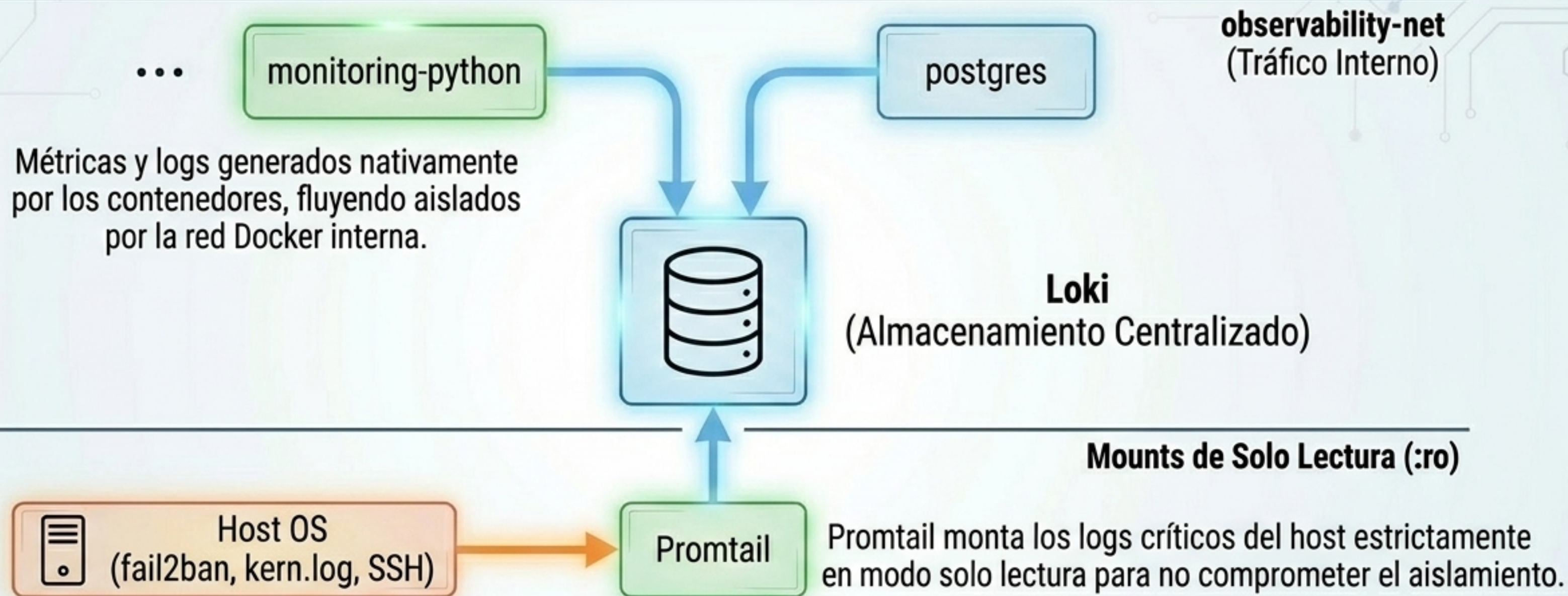
Matriz de Privilegios y Seguridad Runtime

DIAGNÓSTICATABLE

	Clasificación	Usuario (UID)	Capabilities	Acceso docker.sock
Código Propio	Micro-stacks Propios (monitoring-python)	Non-root estricto (UID != 0)	Drop ALL	Bloqueado
Servicios Externos	Infra-Trusted / Upstream (postgres, smtp)	Root permitido (upstream)	Drop ALL	Bloqueado
Observabilidad y Operaciones	Infra-Operacional (observability)	Depende de imagen	Restringidas	Permitido (Solo lectura / Auditado)

Mínimo privilegio pragmático: Seguridad endurecida al máximo para código propio; controles compensatorios (aislamiento de red, volúmenes de solo lectura) para dependencias upstream oficiales.

Modelo Híbrido de Observabilidad



Observabilidad defensiva sin acoplar el runtime. Promtail se trata como un componente de infraestructura confiable.

Estrategia de Resiliencia (Single-Node)

Comportamiento esperado sin orquestador (ADR-0017)

Fallo Interno	Caída de Dependencia	Fallo Humano (docker kill)
		
El proceso interno muere. Docker Compose interviene y reinicia el contenedor automáticamente. Recuperación exitosa.	La conectividad DNS/TCP cae. La app sobrevive en modo degradado y se reconecta automáticamente al volver la DB.	No hay loop de reconciliación tipo K8s. El contenedor NO se recrea automáticamente. Requiere intervención declarativa.

Estrategia basada en la autosuficiencia del servicio y la simplicidad del entorno Single-Node sin orquestador complejo.

Entorno de Desarrollo Normativo (WSL)



Política Estricta: Separación total entre la UI y el entorno de ejecución para garantizar paridad absoluta con el servidor Linux en producción.

Flujo de Implementación a Producción

De cero a operativo en 4 etapas validadas



Validación Continua: Todo cambio valida sintaxis, pruebas E2E y auditoría de seguridad runtime obligatoria antes de ser integrado.